

Levantamento de Requisitos — MVP petland-api

Documento gerado a partir da análise do **Calendário de 7 Semanas (Bootcamp v6)** de Gabriel Falcão da Cruz. Todas as referências são rastreáveis ao calendário original.

1. Visão Geral do MVP

O petland-api é uma **REST API de gestão de petshop** que serve como projeto-âncora do bootcamp. O MVP cobre a **Semana 1 completa** — sendo o produto entregável ao fim do sábado da S1, já testado, containerizado, com cobertura $\geq 80\%$ e publicado no GitHub com Git Flow profissional.

A partir da S2 em diante, o Petland é gradualmente migrado para o ecossistema de microsserviços Nebula-Archive, então o escopo do MVP deve ser fechado antes dessa transição.

Origem das referências no calendário:

- S1 Segunda: *"implementar entidades do Petland como Records", "implementar Singleton, Factory Method, Builder no Petland", "modelar o domínio do Petland no caderno"*
 - S1 Terça: *"Spring Data JPA + DDD — Mapeamento de entidades [...] PostgreSQL", "@Valid nos payloads. @RestControllerAdvice para exceções globais"*
 - S1 Quarta: *"Branches: criar feat/petland-domain", "Dockerfile [...] non-root user", "SonarQube escaneando o Petland, JaCoCo com barreira 80%"*
 - S1 Quinta: *"mover lógica de negócio do Controller para o Domain. Application Service chamando Domain Service", "teste de integração @SpringBootTest com PostgreSQL real em container no Petland"*
 - S1 Sexta: *"JaCoCo no Petland: se $< 80\%$, adicionar testes até a barreira"*
 - S3/S4: *"Testar roteamento completo: POST /api/pets → Gateway → Eureka → App → DB"*
-

2. Escopo do MVP

✓ Dentro do MVP (Semana 1)

- CRUD completo de Pets com entidades em Java Records
- Mapeamento JPA com DDD (Aggregates, Aggregate Roots, Repositories)
- Validação de payloads com @Valid e tratamento global de erros
- Design Patterns implementados: Singleton, Factory, Builder, Strategy, Observer

- Separação em camadas: Controller → ApplicationService → DomainService → Repository
- Testes unitários (JUnit 5 + Mockito) e de integração (Testcontainers + PostgreSQL)
- Cobertura JaCoCo ≥ 80% com Quality Gate bloqueante
- SonarQube local sem issues críticos/blocker
- Dockerfile com non-root user (`eclipse-temurin:21-jre-alpine`)
- `docker-compose.yml` com app + PostgreSQL + Redis + healthchecks
- Spring Actuator + Prometheus + dashboard Grafana (métricas JVM)
- README em inglês com diagrama DDD e badge de cobertura
- Git Flow profissional com Conventional Commits em inglês

✖ Fora do MVP (Fase 2+)

- Spring Cloud Config Server (S2)
- Netflix Eureka Service Discovery (S3)
- Spring Cloud Gateway com filtro JWT (S3)
- Kubernetes / Helm (S3-S6)
- Saga Pattern / CQRS / RabbitMQ (S7)
- CI/CD com GitHub Actions (S6)
- Rate limiting via Redis no Gateway (S3)
- Multi-stage Docker (S2)
- Autenticação/autorização JWT completa (S3)

3. Requisitos Funcionais

RF-01 — Gerenciamento de Pets

Origem: S1 Segunda — *"implementar entidades do Petland como Records. Imutabilidade implícita."*

ID	Descrição	Prioridade
RF-01.1	Cadastrar um novo Pet (nome, espécie, dono)	Must
RF-01.2	Listar todos os Pets cadastrados	Must
RF-01.3	Buscar um Pet por ID	Must
RF-01.4	Atualizar dados de um Pet existente	Must
RF-01.5	Remover um Pet pelo ID	Must

S3/S4 Quarta — "Testar roteamento completo: **POST** */api/pets* → Gateway → Eureka → App → DB"

Endpoints esperados:

```
POST    /api/pets
GET     /api/pets
GET     /api/pets/{id}
PUT     /api/pets/{id}
DELETE  /api/pets/{id}
```

RF-02 — Gerenciamento de Owners (Tutores)

Origem: S1 Segunda — "Entidades, Value Objects" + "modelar o domínio do Petland" + Pet é uma entidade que possui um Owner

ID	Descrição	Prioridade
RF-02.1	Cadastrar um novo Owner	Must
RF-02.2	Buscar Owner por ID	Must
RF-02.3	Listar Pets vinculados a um Owner	Should

RF-03 — Validação de Payloads

Origem: S1 Terça — "@Valid nos payloads. @RestControllerAdvice para exceções globais. ResponseEntity padronizado."

ID	Descrição	Prioridade
RF-03.1	Validar campos obrigatórios (nome, espécie) no cadastro de Pet	Must
RF-03.2	Retornar erro 400 com mensagem legível para payload inválido	Must
RF-03.3	Retornar erro 404 padronizado quando recurso não encontrado	Must
RF-03.4	Retornar erro 500 genérico com mensagem sem stack trace exposto	Must

Formato de resposta de erro padronizado:

```
json
```

```
{
  "timestamp": "2026-06-23T10:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "O campo 'name' é obrigatório",
  "path": "/api/pets"
}
```

RF-04 — Monitoramento e Observabilidade

Origem: S1 Quinta — "Ativar /actuator/prometheus. Prometheus + Grafana em containers. Dashboard JVM."

ID	Descrição	Prioridade
RF-04.1	Expor endpoint <code>/actuator/health</code>	Must
RF-04.2	Expor endpoint <code>/actuator/prometheus</code> com métricas JVM	Must
RF-04.3	Dashboard Grafana com heap, GC, threads e latência de endpoints	Should
RF-04.4	Contador de requisições por endpoint	Should

4. Requisitos Não Funcionais

RNF-01 — Linguagem e Runtime

Origem: S1 Segunda — "Records, Pattern Matching for instanceof, Sealed Classes, Switch Expressions, Text Blocks"

ID	Descrição	Critério de Aceite
RNF-01.1	Java 21 como versão mínima do runtime	<code>java -version</code> retorna 21+
RNF-01.2	Entidades do domínio implementadas como Records	Zero getter/setter boilerplate nas entidades
RNF-01.3	Pattern Matching usado nos type checks do domínio	<code>instanceof</code> sem cast manual
RNF-01.4	Sealed Classes limitando hierarquias (ex: tipos de Pet)	Subtipos restritos explicitamente
RNF-01.5	Text Blocks em queries SQL customizadas	Sem concatenação de string para SQL

RNF-02 — Arquitetura e Design

Origem: S1 Quinta — *"Controller → ApplicationService → DomainService → Repository. Testar separação: ApplicationService não deve importar repositórios diretamente."*

ID	Descrição	Critério de Aceite
RNF-02.1	Separação estrita em 4 camadas: Controller, ApplicationService, DomainService, Repository	Teste de arquitetura ou code review validando imports
RNF-02.2	<code>ApplicationService</code> não importa repositórios diretamente	Ausência de <code>@Autowired Repository</code> em Application Services
RNF-02.3	<code>Controller</code> não contém lógica de negócio	Controllers com no máximo chamadas ao ApplicationService
RNF-02.4	<code>Repository</code> existe por Aggregate Root (DDD)	Um Repository por entidade raiz
RNF-02.5	Singleton: <code>ApplicationConfig</code> gerenciado pelo Spring	Apenas uma instância confirmada por contexto
RNF-02.6	Factory Method: criação de entidades polimórficas via factory	Nenhum <code>new TipoEspecifico()</code> espalhado no domínio
RNF-02.7	Builder: <code>PetBuilder</code> para construção fluente	<code>Pet pet = PetBuilder.create().name(...).species(...).build()</code>
RNF-02.8	Strategy: validações intercambiáveis (regras de agendamento ou tipo de serviço)	Interface <code>ValidationStrategy</code> com implementações distintas
RNF-02.9	Observer: eventos de domínio disparados ao criar/atualizar entidades	<code>ApplicationEventPublisher</code> ou padrão manual de listeners

RNF-03 — Qualidade de Código

Origem: S1 Quarta — "SonarQube escaneando o Petland, JaCoCo com barreira 80%, corrigir 2+ issues críticos" | S1 Sexta — "Rodar JaCoCo no Petland: se < 80%, adicionar testes até a barreira. Badge de cobertura no README."

ID	Descrição	Critério de Aceite
RNF-03.1	Cobertura de código $\geq 80\%$ (JaCoCo)	Build falha se $< 80\%$
RNF-03.2	SonarQube sem issues BLOCKER	<code>sonar.qualitygate.status = OK</code>
RNF-03.3	SonarQube sem issues CRITICAL	Zero issues críticos no scan
RNF-03.4	Duplicação de código $< 3\%$	Sonar Quality Gate configurado
RNF-03.5	Badge de cobertura publicado no README	Link visível no topo do README.md

RNF-04 — Testes

Origem: S1 Terça — *"JUnit 5 + Mockito. assertThrows. cenários de erro. mock de repositório retornando vazio"* | S1 Quinta — *"testes de controller com MockMvc. teste de integração @SpringBootTest com PostgreSQL real em container"*

ID	Descrição	Critério de Aceite
RNF-04.1	Testes unitários com JUnit 5 + Mockito	Todos os use cases cobertos
RNF-04.2	Teste de exceção com <code>assertThrows</code> para cenários de erro	Ex: Pet não encontrado $\rightarrow$ <code>ResourceNotFoundException</code>
RNF-04.3	Mock de repositório retornando vazio para cenário "recurso não existe"	<code>Optional.empty()</code> mockado
RNF-04.4	Testes de controller com MockMvc	HTTP 200/201/400/404 validados por endpoint
RNF-04.5	Teste de integração com <code>@SpringBootTest</code> + PostgreSQL real via Testcontainers	Banco real sobe e persiste no teste
RNF-04.6	<code>docker compose up</code> incluindo banco para testes de integração	Stack de teste isolada do ambiente local

RNF-05 — Containerização

Origem: S1 Quarta — *"Dockerfile manual: FROM eclipse-temurin:21-jre-alpine, WORKDIR /app, COPY, RUN. Non-root user: RUN addgroup -S app && adduser -S app -G app, USER"*

app." | S1 Quarta Compose — *"Redis no Compose [...] Healthcheck no PostgreSQL [...] depends_on [...] service_healthy"*

ID	Descrição	Critério de Aceite
RNF-05.1	<code>Dockerfile</code> com non-root user (grupo <code>app</code> , usuário <code>app</code>)	<code>docker inspect</code> não retorna root como user
RNF-05.2	Base image: <code>eclipse-temurin:21-jre-alpine</code>	Menor footprint possível no MVP
RNF-05.3	<code>.dockerignore</code> excluindo <code>target/</code> , <code>.git</code> , <code>*.md</code>	Imagem sem artefatos de desenvolvimento
RNF-05.4	<code>docker-compose.yml</code> com serviços: app + PostgreSQL + Redis	<code>docker compose up</code> sobe toda a stack
RNF-05.5	Healthcheck no PostgreSQL via <code>pg_isready</code>	<code>test: [CMD-SHELL, pg_isready -U \$POSTGRES_USER]</code>
RNF-05.6	<code>depends_on</code> com <code>condition: service_healthy</code> no serviço <code>app</code>	App só inicia após banco saudável
RNF-05.7	Named Volume para PostgreSQL	<code>docker compose down && up</code> preserva dados
RNF-05.8	Credenciais via <code>.env</code> — zero hardcode no <code>compose.yml</code>	<code>POSTGRES_PASSWORD</code> nunca literal no YAML
RNF-05.9	Redes isoladas no Compose: <code>backend-network</code> e <code>database-network</code>	PostgreSQL inacessível diretamente pelo Redis

RNF-06 — Persistência

Origem: S1 Terça — *"Mapear entidades JPA respeitando Aggregates do DDD. Repository por Aggregate Root."*

ID	Descrição	Critério de Aceite
RNF-06.1	PostgreSQL como banco de produção e integração	Não usar H2 em ambiente não-teste
RNF-06.2	Entidades mapeadas com <code>@Entity</code> respeitando Aggregates do DDD	Um repositório por Aggregate Root
RNF-06.3	Redis no Compose (disponível para cache de sessão)	Container Redis subindo na stack MVP
RNF-06.4	Queries complexas via Text Blocks (Java 21)	Sem concatenação de String para SQL nativo

RNF-07 — Versionamento e Git

Origem: S1 Quarta — *"Branches: criar feat/petland-domain, trabalhar nela, fazer PR simulado. Nunca commitar na main. Rebase interativo [...] Cherry-pick. 10 Conventional Commits no Petland."*

ID	Descrição	Critério de Aceite
RNF-07.1	Nenhum commit direto na branch <code>main</code>	PRs simulados via branches de feature
RNF-07.2	Branches nomeadas como <code>feat/petland-*</code>	Ex: <code>feat/petland-domain</code> , <code>feat/petland-docker</code>
RNF-07.3	Mínimo 10 Conventional Commits em inglês	<code>feat:</code> , <code>fix:</code> , <code>refactor:</code> , <code>docs:</code> , <code>test:</code> , <code>chore:</code>
RNF-07.4	Histórico limpo com rebase interativo antes de merge	<code>git log --oneline</code> sem commits "WIP" ou "fix fix"
RNF-07.5	Tag <code>v1.0.0</code> marcando a entrega do MVP	<code>git tag v1.0.0</code> no commit de fechamento da S1

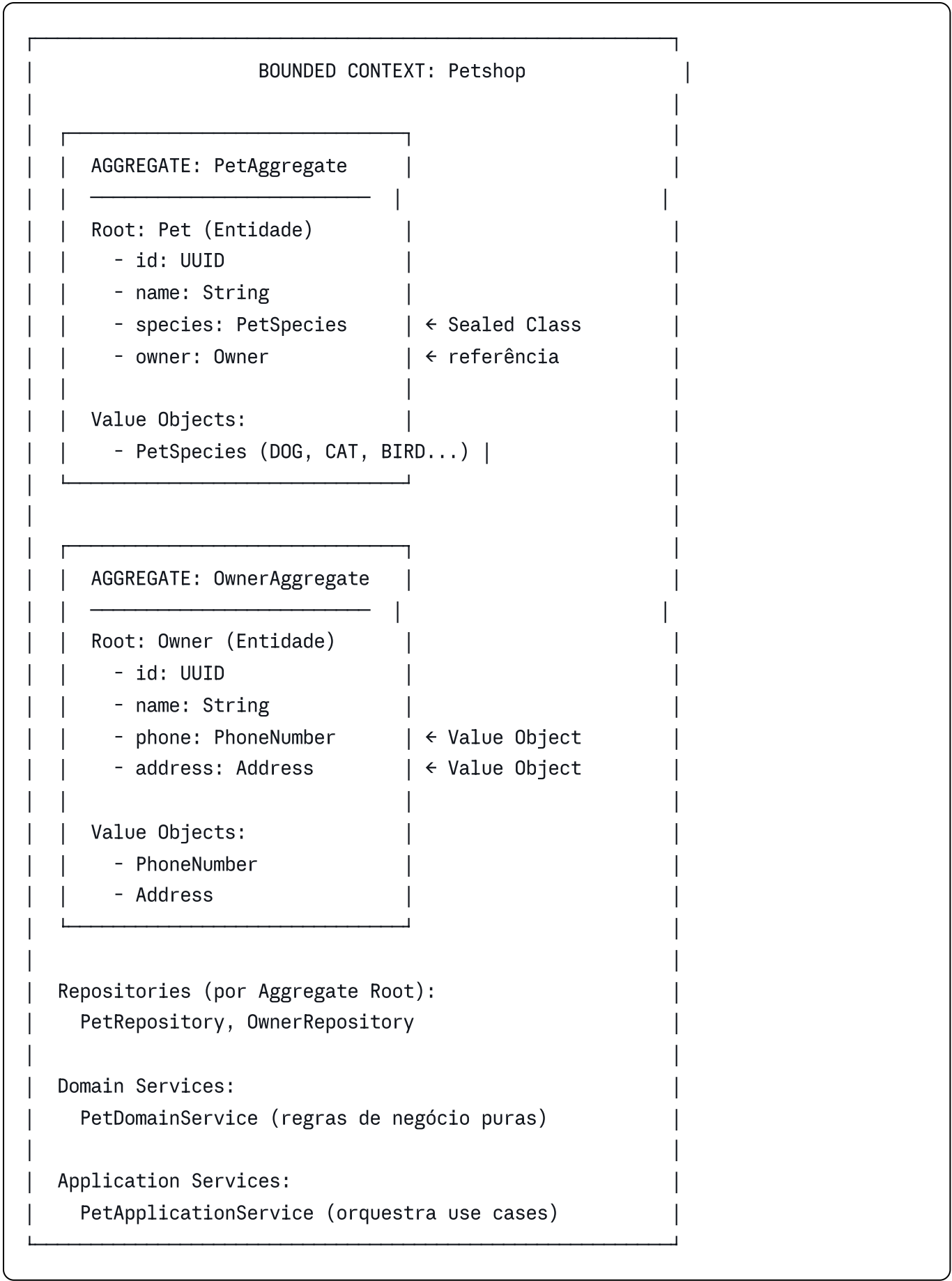
RNF-08 — Documentação

Origem: S1 Sábado — *"Atualizar README do Petland em inglês: diagrama DDD, Design Patterns usados, JaCoCo badge, setup Docker."*

ID	Descrição	Critério de Aceite
RNF-08.1	README em inglês	Arquivo <code>README.md</code> 100% em inglês
RNF-08.2	Diagrama DDD do domínio Petland	Imagem ou ASCII art com Entidades, Value Objects, Aggregates
RNF-08.3	Seção listando os Design Patterns usados e onde foram aplicados	Tabela ou lista no README
RNF-08.4	Badge de cobertura JaCoCo	Shield.io ou JaCoCo badge no topo
RNF-08.5	Instruções de setup Docker (<code>docker compose up</code>)	Seção "How to run" funcional

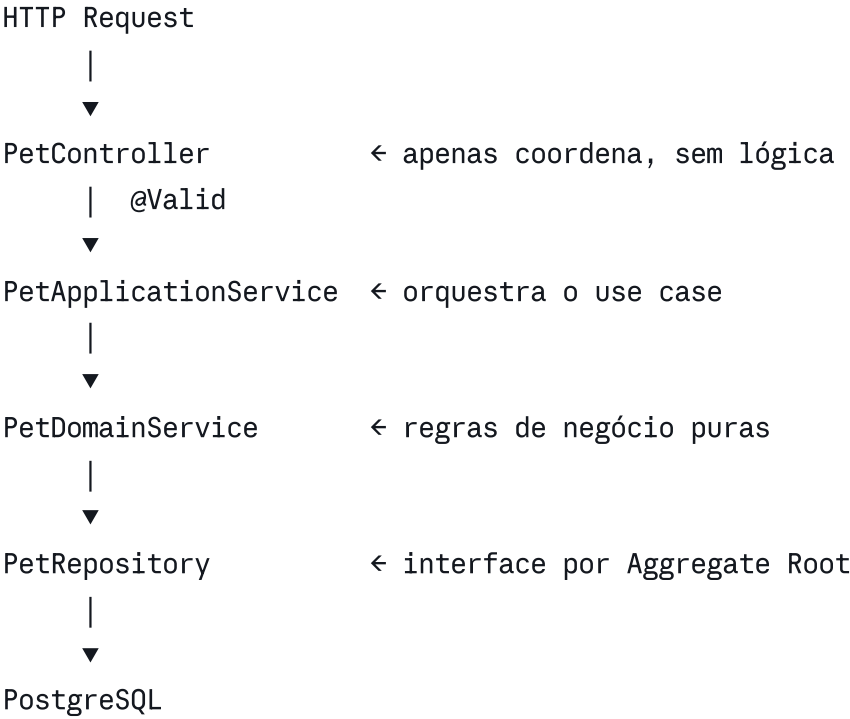
5. Modelo de Domínio DDD (Inferido do Calendário)

Origem: S1 Segunda — *"identificar Entidades, Value Objects e Aggregates"* | S1 Domingo — *"Desenhar diagrama DDD do Petland completo no caderno: entidades, value objects, aggregates, repositórios."*



6. Fluxo de Requisição (Arquitetura de Camadas)

Origem: S1 Quinta — "*Controller → ApplicationService → DomainService → Repository*"



7. Design Patterns — Localização Exata no Código

Origem: S1 Segunda — *"implementar Singleton (ApplicationConfig), Factory Method (entidades polimórficas) e Builder (PetBuilder)"* | S1 Terça — *"refatorar o Petland com Strategy (validações) e Observer (eventos de domínio)"*

Pattern	Onde	Descrição
Singleton	ApplicationConfig	Configuração global gerenciada pelo Spring como bean único
Factory Method	PetFactory	Criação de instâncias polimórficas por tipo de animal sem expor new
Builder	PetBuilder	Construção fluente de Pets com muitos campos opcionais
Strategy	ValidationStrategy	Regras de validação intercambiáveis (ex: por tipo de serviço)
Observer	Domain Events	ApplicationEventPublisher disparando eventos ao criar/atualizar Pets

8. Critérios de Aceite do MVP (Definição de Pronto)

Baseados nos targets do calendário ao final da S1:

- ☐ `POST /api/pets` cadastra um Pet e retorna 201
 - ☐ `GET /api/pets` lista todos os Pets
 - ☐ `GET /api/pets/{id}` retorna 404 para ID inexistente
 - ☐ `PUT /api/pets/{id}` atualiza e retorna 200
 - ☐ `DELETE /api/pets/{id}` remove e retorna 204
 - ☐ Payload inválido retorna 400 com mensagem legível
 - ☐ Entidades implementadas como **Java Records** (sem getter/setter manuais)
 - ☐ Cinco padrões GoF implementados e testáveis isoladamente
 - ☐ Camadas com separação confirmada (ApplicationService não importa Repository)
 - ☐ JaCoCo $\geq 80\%$ — build verde
 - ☐ SonarQube sem issues BLOCKER ou CRITICAL
 - ☐ `docker compose up` sobe app + PostgreSQL + Redis sem erro
 - ☐ Dados do banco persistem após `docker compose down && up`
 - ☐ `/actuator/health` responde `{"status": "UP"}`
 - ☐ `/actuator/prometheus` expõe métricas JVM
 - ☐ README em inglês com diagrama DDD e badge de cobertura publicado
 - ☐ Tag `v1.0.0` criada no GitHub
 - ☐ Histórico do Git com mínimo 10 Conventional Commits em inglês
-

9. Fora de Escopo — Confirmado pelo Calendário

Itens explicitamente postergados para S2+ no calendário:

Item	Semana de Entrada
Spring Cloud Config Server	S2
Docker Multi-stage (redução de imagem)	S2
Netflix Eureka Service Discovery	S3
Spring Cloud Gateway + filtro JWT	S3
Rate limiting (Redis no Gateway)	S3
Kubernetes (Minikube, Deployments, Services)	S3-S4
ConfigMap + Secret K8s	S4
HPA (Horizontal Pod Autoscaler)	S5
Liveness / Readiness / Startup Probes	S5
GitHub Actions CI/CD	S6
Helm Charts	S6
Saga Pattern	S7
CQRS + Views desnormalizadas	S7
ArgoCD / GitOps	Fase 2 (pós-bootcamp)
EKS (cloud managed K8s)	Fase 2
Inglês técnico funcional (pleno)	Fase 2

Levantamento de requisitos extraído do Calendário 7 Semanas · Bootcamp v6 · Junho 2026
Gabriel Falcão da Cruz · Cloud-Native Java Architect track